POS TECH

TECH MANAGEMENT

FASE 4 | AULA 01 -

ARQUITETURAS DA ATUALIDADE E SEUS TRADE-OFFS

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON.....4
SAIBA MAIS 5
MERCADO, CASES E TENDÊNCIAS.....17
O QUE VOCÊ VIU NESTA AULA?.....19

FIAP

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

O QUE VEM POR AÍ?

Nesta jornada, vamos explorar algumas das principais arquiteturas modernas que influenciam diretamente a forma como estruturamos sistemas robustos e escaláveis. Você conhecerá as arquiteturas mais utilizadas atualmente, como hexagonal, onion, clean architecture e vertical slice.

Vamos entender como cada uma delas surgiu, quais problemas buscam resolver e os princípios fundamentais que as sustentam, como o separation of concerns e a inversão de dependência. Ao final desta aula, faremos uma análise comparativa destacando os trade-offs, vantagens e limitações de cada uma destas abordagens.

FIAP

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

HANDS ON

Nesta aula prática, vamos entender a base da evolução das arquiteturas ao longo dos anos. Cada vídeo trará uma explicação sobre o conceito básico de uma arquitetura, seguido de um exemplo prático. Isso nos permitirá visualizar na prática como cada abordagem pode ser aplicada em projetos reais.

Abordaremos a arquitetura hexagonal, padrão proposto por Alistair Cockburn, que introduz o conceito de Portas e Adaptadores, e a Onion Architecture, proposta por Jeffrey Palermo e baseada em anéis concêntricos, com foco na proteção do domínio. Avançaremos para a Clean Architecture, criada por Robert C. Martin, que combina ideias das arquiteturas anteriores com uma divisão mais clara de responsabilidades, e por fim, a Vertical Slice, quebrando a lógica tradicional de camadas.

Encerraremos com uma análise dos pontos fortes, limitações e trade-offs de cada arquitetura. Você verá quando faz sentido aplicar cada abordagem, quais contextos favorecem sua adoção e como tomar decisões arquiteturais mais conscientes e alinhadas ao negócio.

FIAP

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

SAIBA MAIS

Ao desenvolver sistemas modernos, não basta decidir entre monólitos ou microsserviços. A forma como o código é organizado internamente, incluindo a estruturação de domínios, casos de uso e dependências, impacta diretamente na manutenibilidade, testabilidade, escalabilidade e legibilidade da aplicação.

Nesta aula, vamos explorar algumas das arquiteturas mais adotadas na atualidade. Para facilitar o seu entendimento e fornecer uma perspectiva evolutiva, analisaremos cada uma delas com base no período em que foram propostas e no contexto em que surgiram.

Começamos pela **arquitetura hexagonal**, proposta por Alistair Cockburn como uma alternativa à tradicional organização em camadas. Mas, antes de avançarmos, é fundamental compreendermos um dos princípios mais importantes da engenharia de software: o Separation of Concerns (SoC) ou Separação de Conceitos, em português.

Esse princípio visa modularizar uma solução, de forma que cada módulo esteja focado em resolver apenas um problema. No desenvolvimento de software, a Separação de Conceitos permite a divisão de um sistema em diferentes áreas, seções ou módulos, visando alcançar um objetivo principal, que geralmente é um software, uma página da web ou um hardware.

A separação de conceitos é comumente aplicada em softwares de grande porte ou em projetos que são desenvolvidos em mais de uma linguagem de programação. Podemos ver a aplicação desse princípio em um padrão de arquitetura muito conhecido, o Model View Controller (MVC). Nele, encontramos uma divisão clara das responsabilidades em três camadas distintas, cada uma desempenhando um papel específico na arquitetura do código.

A camada View é responsável por apresentar os dados ao usuário, proporcionando a interface com a qual ele interage. Ela se concentra na exibição dos elementos visuais e na interação do usuário com o sistema. Enquanto isso, a camada Model assume a responsabilidade pelo processamento de dados e contém as regras de negócio do sistema. Ela representa a lógica de domínio e o estado da aplicação. Esse é um exemplo de um padrão que utiliza o SoC e que demonstra como ele ajuda a elaborar uma arquitetura flexível, manutenível e sustentável.

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

O SoC foi um dos princípios adotados pelo Dr. Alistair Cockburn, em meados dos anos 90, para proposta da arquitetura hexagonal. Em um artigo na primeira wiki, chamada WikiWikiWeb, ele apresentou essa arquitetura, com alguns princípios cujo foco é modularizar uma solução em três áreas distintas e isoladas:

- Centro do hexágono;
- Lado superior esquerdo, fora do hexágono;
- Lado inferior direito, fora do hexágono;

A figura 1 demonstra essa separação:

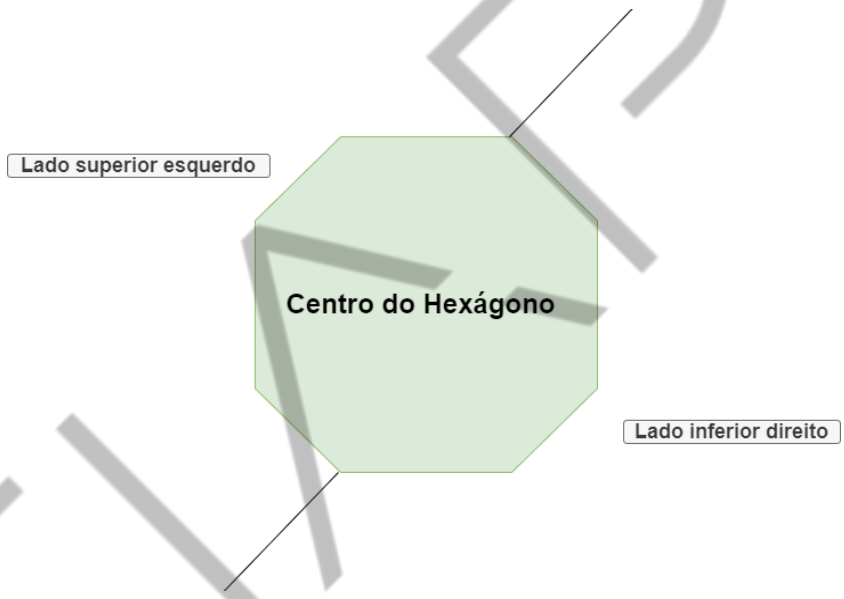

Figura 1 – Exemplo hexágono
Fonte: elaborado pelo autor (2024)

Os objetivos de uma arquitetura hexagonal são parecidos com os de uma arquitetura limpa, que veremos a seguir, visando construir sistemas que favorecem a reusabilidade de código e a alta coesão, possuem baixo acoplamento, independência de tecnologia e facilidade para serem testados.

O quesito de testes é um dos principais pontos para a adoção dessa arquitetura, muito importante para garantir a qualidade e confiabilidade do software. Infelizmente, em algumas abordagens, não é possível realizar todos os testes básicos necessários, mostrados na pirâmide de testes da figura 2, devido ao acoplamento entre camadas e frameworks.

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

Figura 2 – Exemplo pirâmide de testes
Fonte: elaborado pelo autor (2024)

O foco da arquitetura hexagonal é também isolar toda lógica da aplicação no centro do hexágono, tornando esta camada independente de outras camadas ou tecnologias, como o banco de dados utilizado na aplicação ou qual front-end está consumindo as informações.

Consequentemente, mudanças de tecnologia podem ser feitas sem impactar as classes de domínio ou casos de uso, como geralmente são adotados nesta arquitetura, permitindo que eles sejam compartilhados por mais de uma tecnologia.

Por exemplo, um sistema pode ter diversas interfaces (forma de apresentar os dados), como Web, mobile ou uma API, e trocá-las sem impactar a camada de domínio, que fica no centro do hexágono. Isso é perceptível no desenho original desta arquitetura:

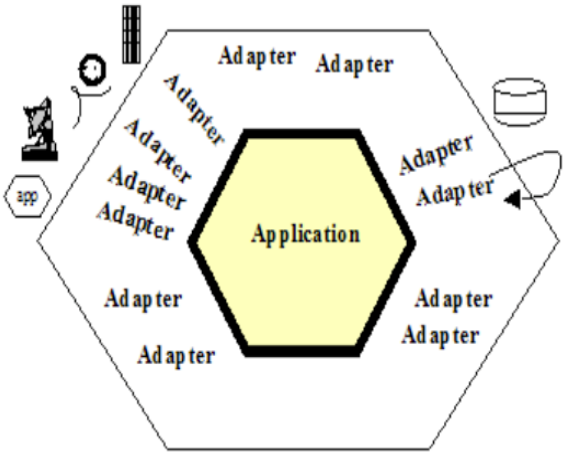

Figura 3 – Arquitetura hexagonal primeiro desenho
Fonte: Cockburn (2005)

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

Note que no centro do hexágono, temos uma camada chamada “Application” (ou aplicação, em português) que não tem relação com a camada de Application da arquitetura de camadas, que veremos mais adiante. A diferença entre elas é o local onde fica o motivo principal do software existir, ou seja, o core ou, como também é conhecido, a camada de domínio, que é o coração do nosso sistema.

Fora do centro, temos os adaptadores, que podem ser vistos como plugues para que as tecnologias fora do hexágono cheguem à camada de aplicação através de algumas portas. Sua função é parecida com a de um adaptador de tomada (popularmente chamados de “benjamin”).

Uma observação interessante é que, apesar de ter nome o "hexagonal", essa arquitetura não especifica diretamente o número seis, nem mesmo o hexágono. A relação está apenas em ser uma figura geométrica: o polígono. Ou seja, poderia ser também representada por um pentágono ou heptágono. A ideia é representar algo que possua diversos lados, que são as diferentes implementações e tecnologias que uma aplicação pode utilizar para expor seus resultados. Portanto, podemos dizer que os lados representam a camada de serviço e suas implementações.

O Dr. Alistair justifica o uso de um hexágono para o nome desta arquitetura afirmando que “Cada face do hexágono representa um motivo pelo qual o sistema deve se comunicar com o mundo exterior. É por isso que são hexágonos concêntricos e não círculos concêntricos” (COCKBURN, apud VALENTE, 2020).

Em 2005, ele tentou renomear sua arquitetura para “arquitetura baseada em Portas e Adaptadores”, com a seguinte justificativa:

> Eu finalmente tive esse momento “aha!” (junho de 2005), no qual vi que as faces do hexágono “interno” são portas e os objetos entre os dois hexágonos são adaptadores e, portanto, o padrão arquitetural consiste em uma arquitetura baseada em portas e adaptadores. Esse nome admite uma explicação melhor do que aquela que eu propus com o nome arquitetura hexagonal (COCKBURN, 2005).

No entanto, essa tentativa de renomear a arquitetura não foi amplamente aceita, e ela continua sendo mais conhecida como arquitetura hexagonal.

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

Onion Architecture

Proposta por Jeffrey Palermo em 2008, a arquitetura onion também surge como uma resposta às limitações das arquiteturas tradicionais em camadas, especialmente no que diz respeito ao acoplamento entre regras de negócio e infraestrutura.

A principal motivação dessa arquitetura é garantir que o domínio da aplicação permaneça protegido de mudanças tecnológicas, organizando o código de forma que as dependências sempre apontem para o centro do sistema, e não o contrário.

Palermo propôs a analogia com a cebola (onion) para representar sua arquitetura em anéis concêntricos: o núcleo mais interno representa o domínio puro, contendo as entidades e regras de negócio, enquanto os anéis externos representam camadas com maior dependência tecnológica, como serviços de infraestrutura, banco de dados, APIs e frameworks.

Esse modelo facilita a manutenção e evolução do sistema, pois alterações nas camadas mais externas (como uma troca de ORM ou API de mensageria) não afetam o núcleo da aplicação. Isso garante maior testabilidade, coesão e isolamento das regras de negócio.

Visualmente, a Onion Architecture se organiza da seguinte maneira:

- Núcleo (Domínio): contém entidades puras e regras de negócio.
- Serviços de Aplicação: define os casos de uso e orquestra a lógica.
- Interfaces (Ports): contratos de entrada e saída.
- Infraestrutura (Adapters): implementações concretas, como controladores REST, repositórios, filas, serviços externos etc.

Assim como na arquitetura hexagonal, aqui também utilizamos o princípio da inversão de dependência para garantir que as implementações concretas (como um repositório que usa PostgreSQL) dependam de contratos definidos no domínio, não o oposto.

Essa abordagem incentiva o desenvolvimento de aplicações robustas e facilmente mantidas através da organização do código em camadas concêntricas, assemelhando-se às camadas de uma cebola.

A figura 4 demonstra essa arquitetura:

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

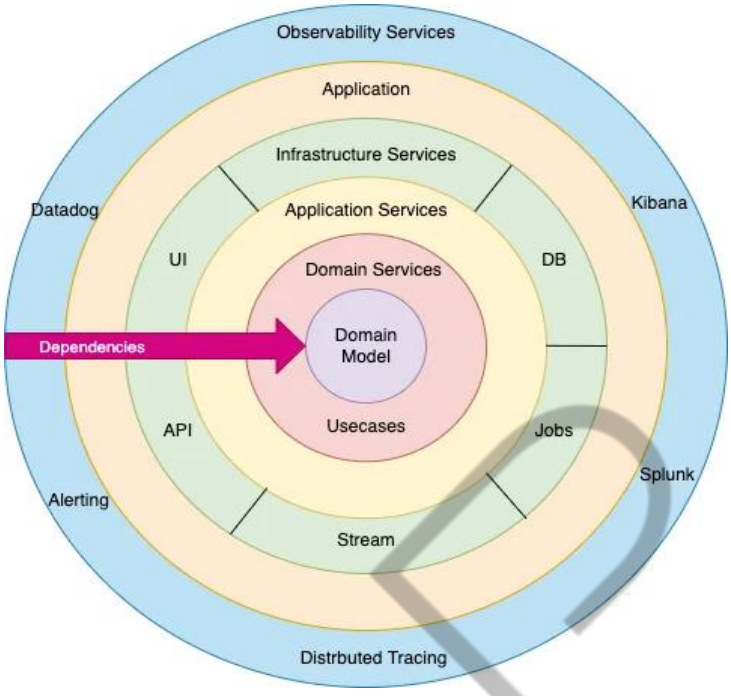

Figura 4 – Exemplo Arquitetura Cebola
Fonte: Medium (2025)

Clean architecture

Proposta por Robert C. Martin, também conhecido como Uncle Bob, em um artigo publicado em seu blog em 2012, este modelo foi posteriormente consolidado em seu livro “Clean Architecture: A Craftsman's Guide to Software Structure and Design”, publicado em 2017.

A clean architecture é, na verdade, uma síntese de boas práticas já presentes em outras arquiteturas, como a onion e a hexagonal. Ela busca oferecer uma estrutura clara, modular e altamente desacoplada, com foco na sustentabilidade do software a longo prazo. Seu objetivo central é permitir que o sistema seja facilmente testável, compreensível e adaptável, mesmo com o passar dos anos e a evolução das tecnologias.

O que motivou a elaboração da clean architecture foi a constatação, por parte de Uncle Bob, de que muitos sistemas acabavam se tornando dependentes das tecnologias escolhidas no início do projeto, como frameworks, bancos de dados e ferramentas de UI, o que gerava um alto custo de manutenção e dificultava a evolução. Sua proposta é proteger a lógica de negócio dessas dependências, colocando-a no centro da arquitetura.

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

Assim como na onion architecture, a clean architecture organiza o código em anéis concêntricos, sendo que as dependências de código devem sempre apontar para o centro. A grande diferença está na nomenclatura e clareza de papéis das camadas, que são bem definidas.

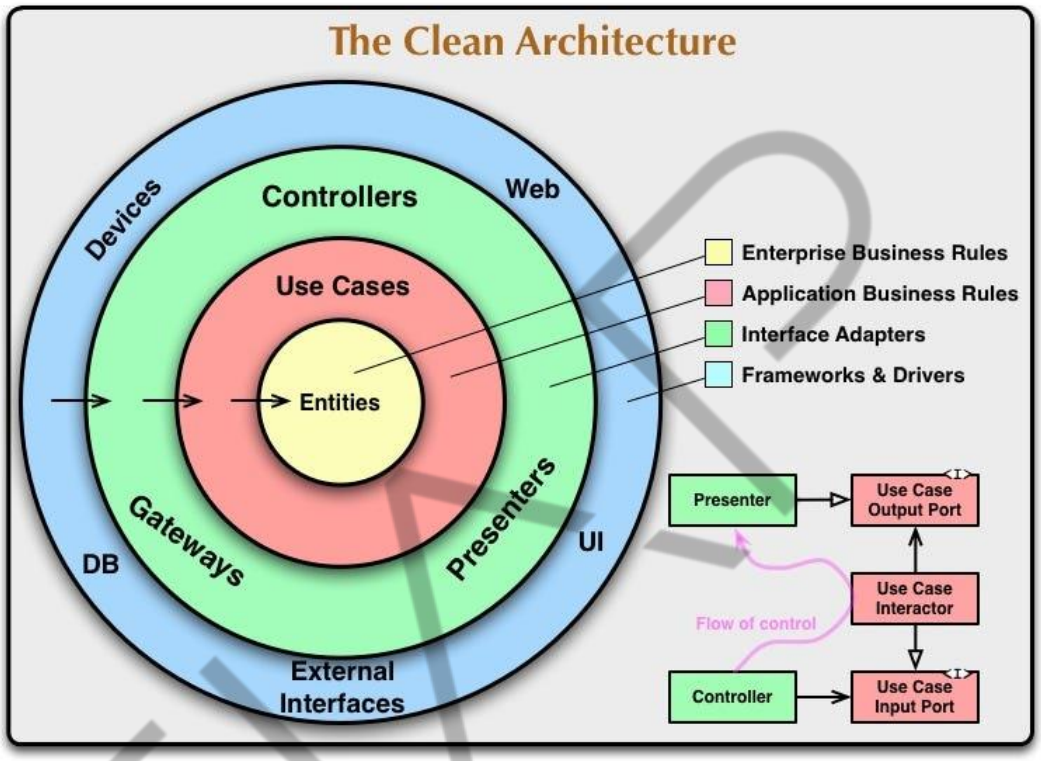

Figura 5 - Clean Architecture
Fonte: Gabriel Fernandes Lemos (2025)

Camadas da clean architecture:

- Entities (Entidades): representam os objetos de domínio e as regras de negócio mais genéricas e reutilizáveis da aplicação. Não dependem de absolutamente nada externo.
- Use Cases (Casos de Uso): contêm as regras específicas da aplicação, orquestrando os fluxos e coordenando entidades e interfaces.
- Interface Adapters (Adaptadores de Interface): traduzem os dados vindos de fora para o formato interno (DTOs, Controllers, Presenters, ViewModels).
- Frameworks & Drivers: tudo aquilo que depende de tecnologia — bancos de dados, bibliotecas, ferramentas de UI, frameworks da web etc.

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

Essa separação respeita fortemente os princípios SOLID, em especial:

- Inversão de Dependência (D): frameworks e ferramentas dependem do domínio, e não o contrário;
- Responsabilidade Única (S): cada camada tem um papel claro e limitado;
- Abstração sobre implementação: contratos e interfaces são priorizados e as implementações vêm depois.

Depois de conhecermos arquiteturas centradas na organização concêntrica do domínio, como a hexagonal, onion e clean architecture, chegamos agora a uma abordagem que propõe uma quebra intencional desses modelos mais rígidos: a **Vertical Slice Architecture**.

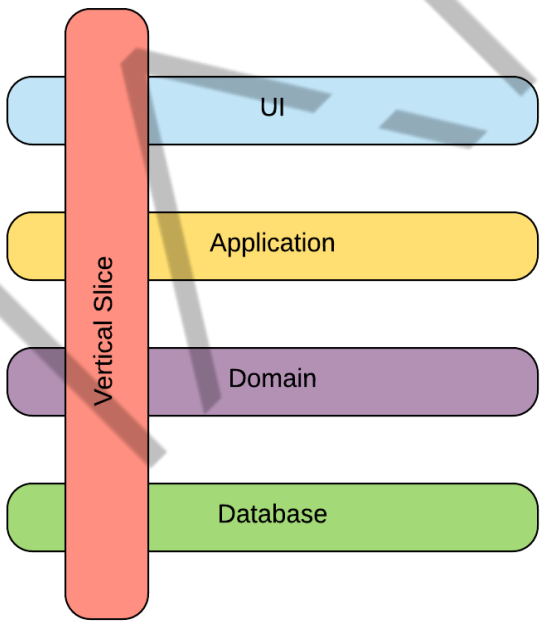

Figura 6 - Vertical Slice
Fonte: Gianty (2024)

Essa arquitetura começou a ganhar força por volta de 2014, especialmente com a popularização do CQRS (Command Query Responsibility Segregation) e dos padrões de feature-based design. Jimmy Bogard, na comunidade .NET, foi um dos principais nomes que ajudaram a difundir e aplicar essa ideia.

A figura 7 demonstra como o CQRS funciona:

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

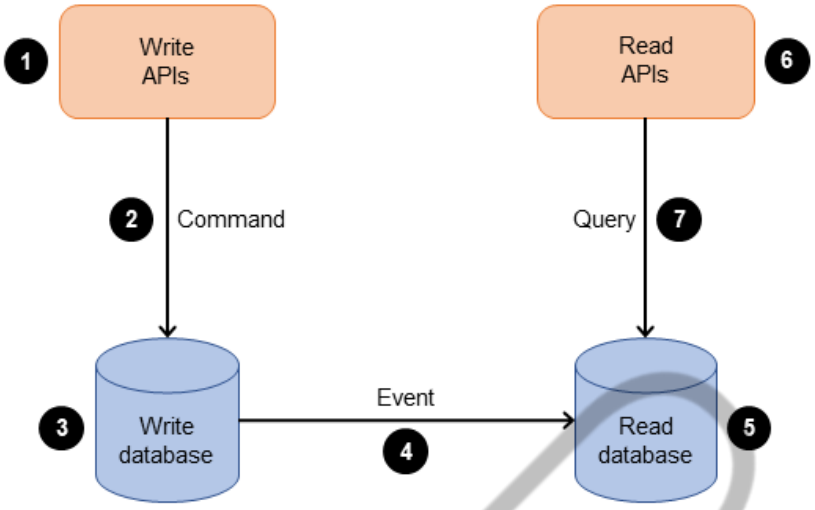

Figura 7 - CQRS

Fonte: AWS (2025)

O CQRS separa as operações de leitura (queries) das operações de escrita (commands) em modelos de dados distintos, permitindo otimizações específicas para cada tipo de operação.

Vamos entender como funciona o CQRS (baseado no diagrama):

Command → Write Database:

- Banco otimizado para escrita (normalizado, com integridade referencial).
- Foco em consistência e validações de negócio.

Event Propagation:

- Após escrita bem-sucedida, eventos são publicados.
- Permite sincronização eventual entre os bancos.

Event → Read Database:

- Eventos atualizam o **Read Database.**
- Permite desnormalização e otimizações para consulta.

Read Database:

- Banco otimizado para leitura (desnormalizado, com índices específicos).
- Pode ter múltiplas projeções dos mesmos dados.

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

Read APIs (Queries):

- Consultas diretas ao banco de leitura.
- Performance otimizada sem joins complexos.

As **vantagens** do CQRS incluem **performance**, com otimização independente para leitura e escrita, **escalabilidade**, pelo scaling independente dos modelos de leitura e escrita, **flexibilidade**, com diferentes tecnologias para cada necessidade e a **simplicidade de queries**, pois dados desnormalizados facilitam consultas.

A vertical slice architecture parte da crítica ao modelo em camadas, que frequentemente leva à criação de projetos com estruturas engessadas, onde tudo é separado por tipo (Controllers, Services, Repositories, Models etc.), e não por funcionalidade real de negócio.

Ao invés de organizar o código em camadas horizontais (por tipo de artefato), a vertical slice organiza o sistema em fatias verticais independentes, onde cada slice representa uma funcionalidade completa, contendo tudo o que é necessário para seu funcionamento: entrada (input), processamento (caso de uso) e saída (output).

Por exemplo, em vez de termos uma pasta Services com dezenas de arquivos, teríamos algo como:

```
/Features
  /CreateOrder
    CreateOrderHandler.cs
    CreateOrderCommand.cs
    CreateOrderValidator.cs
    OrderCreatedEvent.cs
  /GetOrder
    GetOrderQuery.cs
    GetOrderHandler.cs
```

Código-fonte 1 – Estrutura de pastas

Fonte: elaborado pelo autor (2025)

Assim, cada funcionalidade se torna isolada, testável, fácil de entender e com baixíssima dependência das demais. Em sistemas REST, é comum que cada endpoint HTTP represente uma slice. Em aplicações com MediatR, isso se encaixa perfeitamente com os **Handlers** para Commands e Queries.

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

Agora que conhecemos as principais arquiteturas utilizadas na atualidade (hexagonal, onion, clean architecture e vertical slice) é importante reforçar que não existe uma escolha universalmente correta. Cada uma dessas abordagens traz consigo benefícios claros, mas também desafios e limitações que precisam ser cuidadosamente avaliados conforme o contexto do projeto, maturidade do time, requisitos técnicos e objetivos de negócio.

O papel da arquitetura de software não é ditar uma estrutura fixa, mas oferecer diretrizes para que possamos tomar decisões técnicas mais conscientes e sustentáveis. Por isso, ao adotar qualquer uma dessas arquiteturas, é essencial compreender seus trade-offs: o que ganhamos e o que abrimos mão em termos de simplicidade, testabilidade, performance, curva de aprendizado e alinhamento com o domínio do negócio.

Vamos analisar comparativamente os pontos fortes e fracos de cada uma dessas arquiteturas, oferecendo um panorama que pode te ajudar a decidir qual caminho seguir no seu projeto ou na evolução da sua aplicação.

Arquitetura	Principais Benefícios	Principais Desafios / Limitações	Quando Usar
Hexagonal	- Alto desacoplamento - Testabilidade facilitada - Independência de tecnologia - Flexível para múltiplas interfaces	- Curva de aprendizado - Mais abstrações - Overhead em projetos simples	Sistemas que precisam se comunicar com múltiplas interfaces (API, CLI, eventos etc.)
Onion	- Domínio protegido - Organização em camadas concêntricas - Facilita manutenção e testes - Segue SOLID	- Complexidade inicial - Conceitos podem ser mal interpretados	Aplicações de médio a grande porte com lógica de negócio rica e múltiplas integrações

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

Clean Architecture	- Clareza na separação de responsabilidades - Independência de frameworks - Orientada a testes - Evolutiva	- Verbosidade - Maior esforço 0inicial de estruturação	Projetos com expectativa de longo prazo, mudanças tecnológicas e alta exigência de qualidade
Vertical Slice	- Organização por funcionalidade real - Testes mais simples - Ideal para CQRS - Baixo acoplamento	- Risco de duplicação - Pode gerar inconsistência entre slices se mal implementado	APIs modernas, microsserviços, sistemas orientados a casos de uso

Tabela 1 - Tabela Comparativa com trade-offs

Fonte: elaborado pelo autor (2025)

Portanto, se o objetivo é maximizar a coesão do domínio e desacoplar infraestrutura, hexagonal e onion são excelentes escolhas. Se você precisa de uma arquitetura longeva, flexível e altamente testável, a clean architecture oferece uma visão robusta e consolidada. Já para aplicações que demandam entregas rápidas, com organização por caso de uso e baixo acoplamento entre features, a vertical slice architecture pode oferecer o melhor equilíbrio.

A arquitetura ideal não é aquela que está na moda, mas sim a que atende ao seu contexto com responsabilidade técnica e clareza de propósito.

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

MERCADO, CASES E TENDÊNCIAS

O mercado de desenvolvimento de software tem demonstrado uma evolução significativa na adoção de arquiteturas modernas, com empresas buscando soluções que ofereçam flexibilidade, manutenibilidade e escalabilidade. As arquiteturas hexagonal, clean, onion e vertical slice têm se destacado como padrões dominantes para diferentes contextos empresariais.

Arquitetura hexagonal: cases de mercado

Muitas empresas de streaming têm casos emblemáticos de sucesso com a arquitetura hexagonal. Elas utilizam este padrão em seus Estúdios Workflows Teams para gerenciar sistemas que cruzam múltiplos domínios de negócio. Isso resolve o problema da necessidade de trocar fontes de dados sem impactar a lógica de negócio.

- Resultado: conseguir migrar de JSON API para GraphQL em apenas 2 horas.
- Benefício: capacidade de trocar integrações com uma única linha de código.
- Cenários de adoção no mercado: sistemas de integração empresas em processo de modernização de sistemas legados.

Clean Architecture - Domínio Enterprise

Com uma adoção corporativa massiva, a clean architecture tem se tornado o padrão ouro para sistemas enterprise complexos. Cases conhecidos:

- Empresas de grande porte: bancos, seguradoras e multinacionais.
- Sistemas bancários: para isolamento de regras de negócio críticas.
- Plataformas SaaS Enterprise, onde a lógica de domínio é complexa.

Onion Architecture - Sistemas Domain-Driven

A Onion Architecture tem encontrado seu nicho em sistemas altamente orientados ao domínio: Setores de adoção incluem saúde, utilizada em sistemas de gestão hospitalar e prontuários eletrônicos, logística, em sistemas de rastreamento e otimização de rotas, e manufatura, no controle de processos industriais.

Cases documentados:

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

- Sistemas ERP customizados para grandes corporações.
- Plataformas de gestão com regras de negócio complexas.
- Aplicações de conformidade em setores regulamentados.

Vertical Slice Architecture - A revolução CQRS

Jimmy Bogard, criador do padrão, tem visto uma adoção crescente da arquitetura, especialmente em startups de tecnologia, plataformas de e-commerce (para features independentes e deploy contínuo) e sistemas de microsserviços, especialmente com CQRS (Command Query Responsibility Segregation).

Fatores de decisão no mercado

Hexagonal: escolhida quando há alta necessidade de integrações e mudanças frequentes de fornecedores externos.

Clean: preferida para sistemas enterprise complexos onde a longevidade e manutenibilidade são críticas.

Onion: selecionada para domínios altamente especializados com regras de negócio intrincadas.

Vertical Slice: adotada em ambientes ágeis com necessidade de entrega rápida e contínua.

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos algumas das principais arquiteturas modernas que influenciam diretamente a forma como estruturamos sistemas robustos e escaláveis. Você conheceu arquiteturas como hexagonal, onion, clean architecture e vertical slice, entendendo como cada uma delas surgiu, quais problemas buscaram resolver e quais os princípios fundamentais que as sustentam, como o Separation of Concerns e a Inversão de Dependência.

Ao final, realizamos uma análise comparativa, destacando os trade-offs, vantagens e limitações de cada abordagem.

FIAP

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

REFERÊNCIAS

BOGARD, J. **Vertical Slice Architecture – Feature-Based Thinking.** 2016. Disponível em: https://jimmybogard.com/vertical-slice-architecture/. Acesso em: 21 jul. 2025.

COCKBURN, A. **Arquitetura hexagonal : Ports and Adapters.** 2005. Disponível em: https://alistair.cockburn.us/hexagonal-architecture/. Acesso em: 21 jul. 2025.

FERNANDES, G. **Clean Architecture: Uma abordagem baseada em princípios.** 2019. Disponível em: https://medium.com/@gabrielfernandeslemos/clean-architecture-uma-abordagem-baseada-em-princ%C3%ADpios-bf9866da1f9c. Acesso em: 21 jul. 2025.

VALENTE, M. T. **Engenharia de Software Moderna**. Belo Horizonte: UFMG, 2020.

FIAP

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

PALAVRAS-CHAVE

Palavras-chave: Union, Clean Arch, Hexagonal, Vertical Slice

FIAP

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com

POS TECH

PDF exclusivo para Aevilés de Aguiar Silva - rm367851
aeviles@hotmail.com